

MC852

Programação paralela

Ivan Freitas, RA 093887

I. INTRODUÇÃO

A LEI de Moore manteve-se verdadeira por muitos anos, permitindo um rápido crescimento da capacidade computacional disponível no mercado. No entanto, esse crescimento está limitado atualmente por diversos fatores: limite de tamanho na miniaturização de componentes, aumento no consumo de energia de processadores, grande dissipação de energia na forma de calor, entre outros.

Além disso, muitas aplicações específicas possuíam características que permitiam que o desempenho das mesmas se beneficiasse ao aproveitar todos os recursos de execução paralela existentes, tais como máquinas SIMD, MIMD, VLIW, *pipelining*, etc. Conforme os computadores foram tornando-se cada vez mais acessíveis, a opção mais óbvia para melhorar o desempenho dessas aplicações foi agregar múltiplas unidades de processamento em uma única máquina, ou chip, ou ainda interconectar várias máquinas e distribuir o processamento entre elas.

Diversas arquiteturas e técnicas de programação surgiram para explorar o paralelismo presente nesses diversos problemas, e que não é nativo da arquitetura de von Neumann. Este trabalho busca mostrar as diferentes técnicas de programação, bem como sua aplicabilidade e relação com as arquiteturas de hardware disponíveis no momento.

Este artigo está organizado na seguinte maneira: conceitos sobre a arquitetura fundamental estão esclarecidos na seção II, na seção III há a exposição dos modelos de programação mais comuns, aplicações desses modelos são discutidas na seção IV e, por fim, a conclusão do trabalho encontra-se na seção V.

II. CONCEITOS

A. Arquitetura von Neumann

A arquitetura de von Neumann [1] é uma arquitetura de computador que segue o conceito

de computador de programa armazenado (*stored-program computer*), onde o código do programa e os dados residem no mesmo componente, a memória. Essa arquitetura descreve um computador composto por uma unidade de processamento (unidade lógica e aritmética e registradores), mecanismos de entrada e saída, memória, armazenamento externo e unidade de controle.

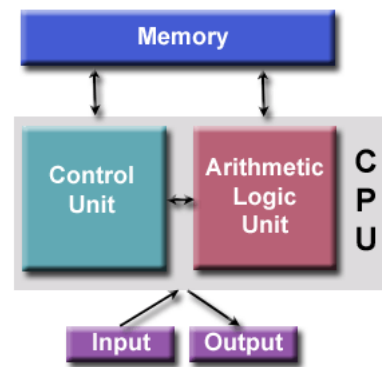


Fig. 1. Diagrama da arquitetura de von Neumann

As arquiteturas atuais se baseiam na de von Neumann, com modificações. Uma característica evidente dessa arquitetura é que foi concebida considerando-se um único fluxo de execução [1], ou seja, não levou em conta a possibilidade de programas executando em paralelo. Além disso, com a separação entre a unidade de processamento e a memória, surge o gargalo da arquitetura de von Neumann [2], onde o desempenho acaba sendo limitado à velocidade de comunicação entre o processador e a memória.

B. Arquitetura de memória

Com o advento da programação paralela, diversas modificações surgiram na arquitetura de von Neumann, como a utilização de múltiplos processadores, múltiplos *cores* por processador e diferentes organizações da memória principal. Essa última mudança possui nomenclatura própria, definida a seguir:

a) *UMA: Unified Memory Access* [3], onde os processadores acessam a memória uniformemente, i.e., todos os processadores enxergam a memória da mesma maneira. Esse tipo de arquitetura é implementado com um dispositivo de memória principal central, geralmente acessado pelos diversos processadores por meio de um barramento ou *crossbar switch*. Cada processador pode, ainda, manter uma *cache* local.

b) *NUMA: Non-Uniform Memory Access* [3], onde cada unidade de processamento inclui (além de, possivelmente, *caches* locais), dispositivos de memória principal, e todos esses dispositivos são acessíveis por todos os processadores. Dessa forma, um processador consegue acessar rapidamente dados da sua memória principal local, mas a de outras unidades de processamento precisa de comunicação via um barramento diferente ou rede.

Comparando-se as arquiteturas de memória, UMA é mais simples de ser implementada e utilizada, os problemas de sincronização são resolvidos mais rapidamente, porém possui menor escalabilidade para expansão do sistema. Já NUMA, é de mais complexa implementação, pois os problemas de sincronização são mais complexos e demorados, porém a expansão do sistema é mais fácil.

NUMA é amplamente utilizada devido à acessibilidade de criação de *commodity clusters*, sistemas paralelos compostos pela interconexão de vários computadores menores, comuns no mercado.

III. MODELOS DE PROGRAMAÇÃO

Dentre diversas estratégias de desenvolvimento, as seguintes foram escolhidas como exemplo, por serem usadas em diversos problemas com ampla aplicação prática.

A. Shared Memory

Shared memory é o modelo onde a sincronização entre diferentes unidades de processamento é feito por leituras ou escritas em regiões compartilhadas de memória. O suporte em hardware existe para garantir a coerência entre a memória e as *caches* dos processadores, e disponibilizar mecanismos eficientes para comunicação (evitando busy-waiting, por exemplo).

Isso implica na existência de sequências de operações que podem gerar resultados incoerentes, além

de um gargalo de desempenho caso a região compartilhada de memória seja acessada com frequência. Os problemas de coerência são resolvidos com a implementação de mecanismos de sincronização em hardware.

Problemas cujas partes independentes comunicam-se poucas vezes (em relação ao total da execução) são os principais candidatos a usarem esse modelo de programação. Com poucos acessos à memória compartilhada, o gargalo de performance tem pouco impacto no desempenho geral do programa, principalmente em sistemas com arquitetura de memória UMA. Ainda, memória compartilhada é um dos modelos mais simples do ponto de vista do programador, pois a sincronização se resume a leituras ou escritas de memória, mais código que verifique a consistência durante a execução de operações que dependem dos valores compartilhados.

B. Message Passing

O modelo de *message passing*, ou passagem de mensagens, é aquele em que entidades (processos, *threads*) que executam independentemente trocam informações entre si através do envio de um conjunto de informações denominado mensagem, através de algum barramento ou rede de comunicação.

Nesse modelo surgem novos desafios. O envio de mensagens depende de uma convenção de comunicação, a qual é arbitrária, no sentido que quaisquer aplicações podem definir e seguir a convenção que lhes parecer mais adequada. Ainda, na comunicação entre as entidades, é necessário algum recurso que permite a nomeação de emissores e receptores, já que as entidades podem mudar sua localização. E por fim, existe ainda a discriminação entre passagem de mensagens síncronas e assíncronas, cada uma com suas vantagens e desvantagens, cabendo ao programador a decisão sobre qual tipo usar.

Quando à questão da nomeação, esta em geral ocorre por identificadores únicos, como PID (*process identifier*) ou endereços de rede. Os protocolos de comunicação foram sendo desenvolvidos ao longo dos anos, hoje predominando o protocolo MPI (*Message Passing Interface*) [4], disponível para uso em várias linguagens de programação.

Com todas essas camadas de abstração, passagem de mensagens acaba tendo um *overhead* grande. No entanto, do ponto de vista do programador, seu uso não é tão complexo, dado que bibliotecas

relativas aos protocolos e suporte por parte do sistema operacional existam. Essa solução também apresenta grande escalabilidade, pois é genérica e não pressupõe a existência de um recurso compartilhado (exceto pela rede ou barramento). Portanto, é ideal para aplicação em sistemas com grande quantidade de entidades executando distribuídamente, como sistemas com arquitetura de memória NUMA.

C. Data parallelism

Data parallelism se aproveita uma característica específica de determinados problemas: o resultado final pode ser obtido pela composição de computações, independentes entre si, de subpartições dos dados de entrada. Sendo assim, os dados são particionados e cada unidade de processamento fica responsável pela computação do resultado em relação à uma dessas partições, e ao final todos os resultados parciais são combinados no resultado final. Essa independência dos resultados parciais permite que eles sejam executados em paralelo.

As desvantagens desse modelo são, a especificidade da sua aplicação, sendo útil apenas em problemas que apresentem certas características, e gargalos quando os dados são particionados para o início da computação e quando os resultados parciais são recompostos no resultado final, principalmente devido à largura de banda de comunicação limitada.

Os problemas que conseguem se beneficiar de *data parallelism* conseguem, teoricamente, grande ganho de desempenho [5]. Com o advento de GPU's com grande quantidade de unidades de processamento capazes de operar em paralelo, *data parallelism* vem tornando-se uma alternativa cada vez mais viável [6]. Ainda existe uma variação denominada *nested data parallelism* [7] [5], muito útil para resolução de problemas com algoritmos do tipo *divide-and-conquer*, que não espera que as computações parciais sejam as mesmas.

IV. APLICAÇÕES DOS MODELOS DE PROGRAMAÇÃO

Nesta seção modelos de programação serão exemplificados e discutidos.

A. Data parallelism com Haskell

Haskell [8] é uma linguagem de programação puramente funcional muito utilizada em pesquisas

acadêmicas, incluindo pesquisas relacionadas à computação paralela [9] [7]. Dentre suas características estão *lazy evaluation*, ou seja, a avaliação de expressões só ocorre quando estas são usadas e a inexistência de efeitos colaterais em funções, que a torna uma linguagem interessante para paralelismo de dados.

Listing 1. Cálculo do produto escalar entre dois vetores

```
dotp :: Num a => [a] -> [a] -> a
dotp xs ys = sumP [x * y | x <- xs | y <- ys]
```

O exemplo acima mostra o cálculo do produto interno entre dois vetores utilizando paralelismo de dados em Haskell. Note que o paralelismo é implícito: cada uma das multiplicações está sendo feita em paralelo (no máximo o número de processadores ou *cores* por vez), e ao final tudo é somado para obter o resultado final.

B. Message passing com MPI

Observe o exemplo abaixo, mostrando uma comunicação, bloqueante, via passagem de mensagens utilizando MPI [10].

Listing 2. Comunicação bloqueante com MPI

```
#include "mpi.h"
#include <stdio.h>

int main(argc, argv)
int argc;
char *argv[]; {
    int numtasks, rank, dest, source, rc, count, tag=1;
    char inmsg, outmsg='x';
    MPI_Status Stat;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        dest = 1;
        source = 1;
        rc = MPI_Send(&outmsg, 1, MPI_CHAR,
                      dest, tag, MPI_COMM_WORLD);
        rc = MPI_Recv(&inmsg, 1, MPI_CHAR,
                      source, tag, MPI_COMM_WORLD, &Stat);
    }

    else if (rank == 1) {
        dest = 0;
        source = 0;
        rc = MPI_Recv(&inmsg, 1, MPI_CHAR,
                      source, tag, MPI_COMM_WORLD, &Stat);
        rc = MPI_Send(&outmsg, 1, MPI_CHAR,
                      dest, tag, MPI_COMM_WORLD);
    }

    rc = MPI_Get_count(&Stat, MPI_CHAR, &count);
```

```
printf("Task %d: Received %d char(s)"
      "from task %d with tag %d \n",
      rank, count, Stat.MPI_SOURCE, Stat.MPI_TAG);

MPI_Finalize();
}
```

Observe que existe um *overhead* durante a inicialização de parâmetros utilizados na comunicação, além do *overhead* da criação e passagem das mensagens em si.

Ainda, observe que o programa permanece bloqueado enquanto espera por uma mensagem. Essa é a característica de uma comunicação bloqueante, na qual o processo fica em *busy waiting* esperando pela chegada da mensagem.

V. CONCLUSÃO

Paralelismo é um dos grandes desafios da computação, que voltou a ganhar importância nos últimos anos com a abundância e barateamento de máquinas *commodity* permitindo a construção de *clusters* e o advento de GPGPU's capazes de processamento paralelo em massa, permitindo que problemas cada vez maiores possam ser resolvidos eficientemente e com custo menor.

No entanto, tanto arquiteturas de hardware como técnicas de programação tiveram de evoluir para acompanhar esse ganho massivo em paralelismo, pois técnicas automatizadas de paralelismo ainda estão longe de resolver todos os problemas existentes com desempenho aceitável, cabendo ao programador analisar cada situação.

A programação paralela depende do entendimento do problema a ser resolvido, aliado à uma estratégia de paralelismo restrita aos limites dos recursos disponíveis, e suporte em hardware. Este trabalho mostrou três modelos de programação paralela diferentes, considerando suas vantagens e desvantagens.

REFERENCES

- [1] J. von Neumann, "First draft of a report on the edvac," *Annals of the History of Computing, IEEE*, vol. 15, no. 4, pp. 27–75, 1993.
- [2] J. Backus, "Can programming be liberated from the von neumann style?: a functional style and its algebra of programs," *Commun. ACM*, vol. 21, no. 8, pp. 613–641, Aug. 1978. [Online]. Available: <http://doi.acm.org/10.1145/359576.359579>
- [3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design, Fourth Edition, Fourth Edition: The Hardware/Software Interface (The Morgan Kaufmann Series in Computer Architecture and Design)*, 4th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2008.
- [4] W. Gropp, E. Lusk, N. Doss, and A. Skjellum, "A high-performance, portable implementation of the mpi message passing interface standard," *Parallel Comput.*, vol. 22, no. 6, pp. 789–828, Sep. 1996. [Online]. Available: [http://dx.doi.org/10.1016/0167-8191\(96\)00024-5](http://dx.doi.org/10.1016/0167-8191(96)00024-5)
- [5] W. D. Hillis and G. L. Steele, Jr., "Data parallel algorithms," *Commun. ACM*, vol. 29, no. 12, pp. 1170–1183, Dec. 1986. [Online]. Available: <http://doi.acm.org/10.1145/7902.7903>
- [6] D. Tarditi, S. Puri, and J. Oglesby, "Accelerator: using data parallelism to program gpus for general-purpose uses," in *in Proceedings of the 12th international conference on Architectural*, 2006, pp. 325–335.
- [7] M. M. T. Chakravarty, R. Leshchinskiy, S. P. Jones, G. Keller, and S. Marlow, "Data parallel haskell: a status report," 2007.
- [8] "Haskell programming language," <http://www.haskell.org/haskellwiki/Haskell>.
- [9] "Data parallel haskell," http://www.haskell.org/haskellwiki/GHC/Data_Parallel_Haskell.
- [10] "Mpi tutorial," <https://computing.llnl.gov/tutorials/mpi/>.